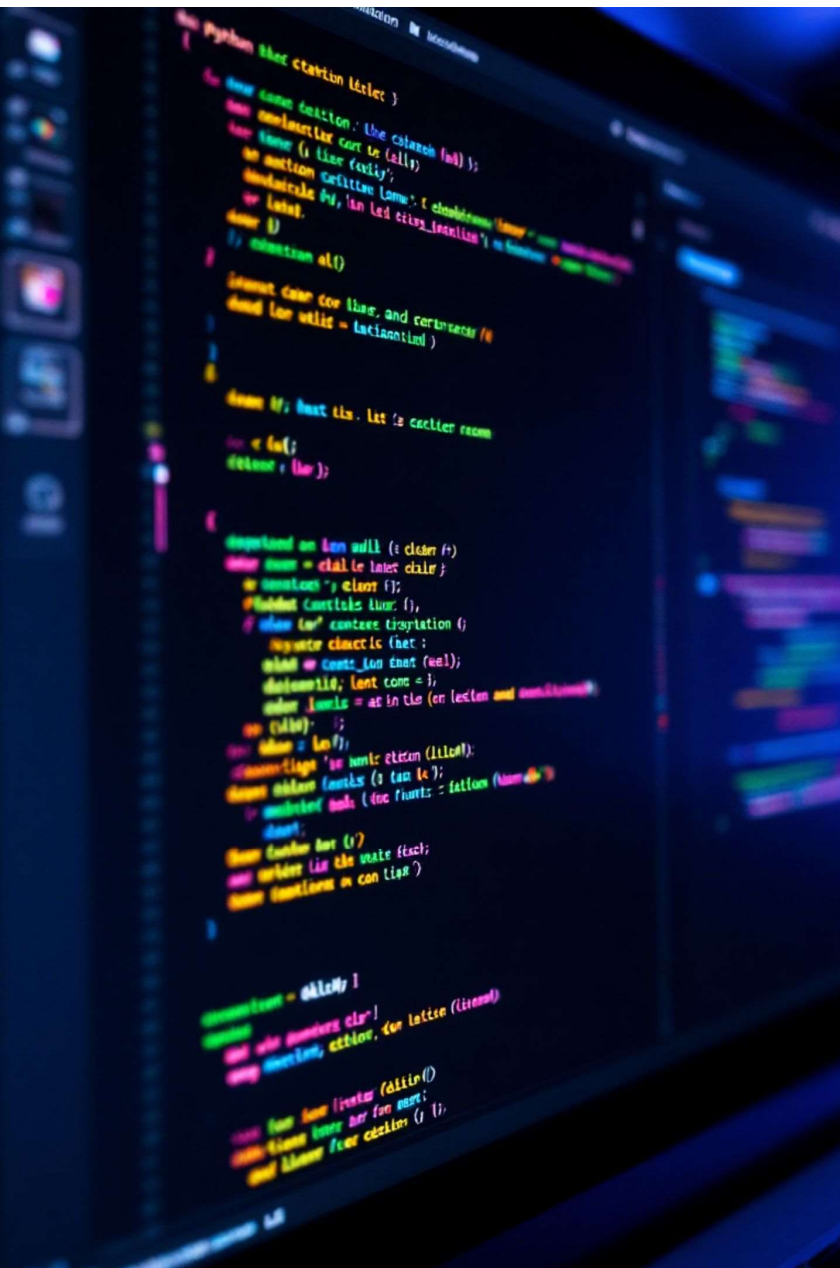




Fundamentos de Python: Sintaxis y Operadores

Python es un lenguaje interpretado que ejecuta el código línea por línea, con una sintaxis clara y legible que lo hace ideal tanto para principiantes como para desarrolladores experimentados.

Sintaxis Básica e Indentación

Sintaxis Básica

El código en Python se divide en líneas, que llamamos sentencias y que son instrucciones ejecutables por el intérprete. Cada sentencia contiene operaciones, estructuras de control o manipulación de datos.

Ejemplo: **x = 5** es una sentencia de asignación que asigna el valor 5 a la variable x.

Indentación

En Python, **la indentación define la estructura y los bloques de código**, a diferencia de otros lenguajes que utilizan llaves {}.

La convención más común es utilizar 4 espacios para cada nivel de indentación, manteniendo consistencia en todo el código (usar siempre e mismo formato).

Ejemplo:

```
#Declaración de variables
x = 5
y = 10

#Definición de función
def suma(a, b):
    # Bloque de código indentado
    resultado = a + b
    return resultado

#Estructura de control
if x < y:
    # Bloque de código indentado
    print("x es menor que y")
else:
    # Otro bloque indentado
    print("y es menor que x")
```

```

reserved {
def _return = "def, return, an complat; roday"if"))
def
(anciow" = Iwate)
def rewium, {
    renur return,_nost "* cole)) {
        def return"if atther, return, ixel(("elses")
        if _otton = racties)
        wf
        "arppla" = dctp
def
    reserved
        return elsec elses;
    }
}

```

Comentarios y Palabras Reservadas

Comentarios

Los comentarios en Python se crean utilizando el símbolo `#`. Todo lo que sigue a este símbolo en la misma línea es tratado como un comentario y no se ejecutará.

#Esto es un comentario en Python

Son útiles para hacer que el código sea más comprensible para otros programadores o para ti mismo en el futuro.

Palabras Reservadas

Son términos con significado específico reservados por el lenguaje para su uso interno. No pueden utilizarse como identificadores (nombres de variables, funciones, etc.).

Ejemplos: False, None, True, and, as, assert, async, await, break, class, continue, def, del, elif, else, except, finally, for, from, global, if, import, in, is, lambda, nonlocal, not, or, pass, raise, return, try, while, with, yield.

Operadores Aritméticos



Suma (+)

Suma dos números o concatena cadenas y listas. Ejemplo:

`a = 5`

`b = 3`

`resultado = a + b` #resultado es 8



Resta (-)

Resta un número de otro.

Ejemplo:

`a = 10`

`b = 7`

`resultado = a - b` #resultado es 3



Multiplicación (*)

Multiplica dos números.

Ejemplo:

`a = 5`

`b = 4`

`resultado = a * b` #resultado es 20



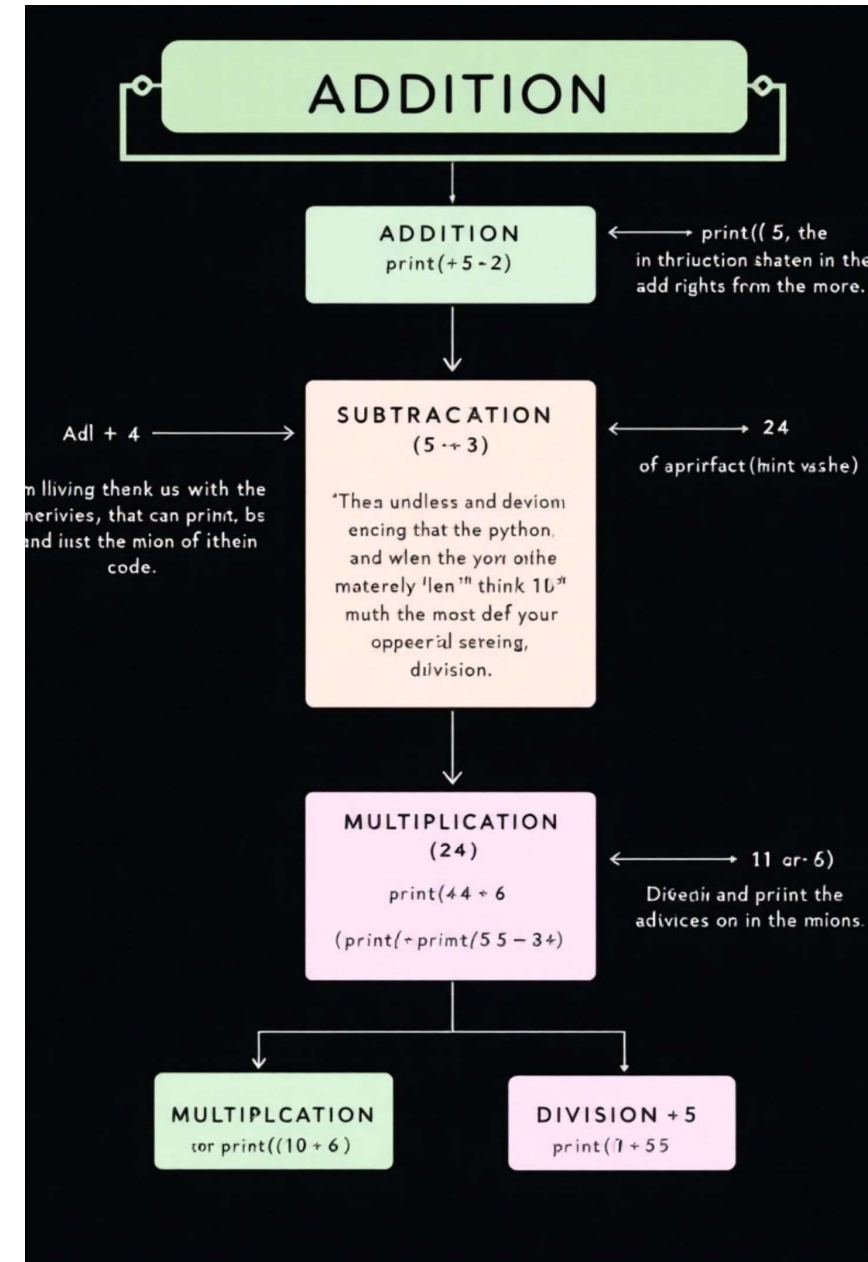
División (/)

Divide un número entre otro. En Python 3, siempre devuelve un número de punto flotante (float).

Ejemplo: `a = 10`

`b = 2`

`resultado = a / b` #resultado es 5.0



Más Operadores Aritméticos

%

Módulo (%)

Obtiene el residuo de una división entre dos números.

Ejemplo:

a = 10,

b = 3,

resultado = a % b #resultado es 1



División Entera (//)

Divide un número entre otro y devuelve el cociente entero, descartando cualquier parte decimal.

Ejemplo:

a = 10

b = 3

resultado = a // b #resultado es 3

X¹

Potencia (**)

Eleva un número a una potencia.

Ejemplo: a = 2

b = 3

resultado = a ** b #resultado es 8

a = 10

modulo = a */* b ## result: 1

modulo = a // b: 3

floor division 1 # a * b + 3

expentiation # result: 1000

$\begin{array}{r} 10 \\ - 8 \\ \hline 2 \end{array}$	8
Resto de la división entera	1 Cociente de la división entera

Otro ejemplo con todos los operadores aritméticos juntos:

`a = 12`

`b = 5`

`suma = a + b` `# suma = 17`

`resta = a - b` `# resta = 7`

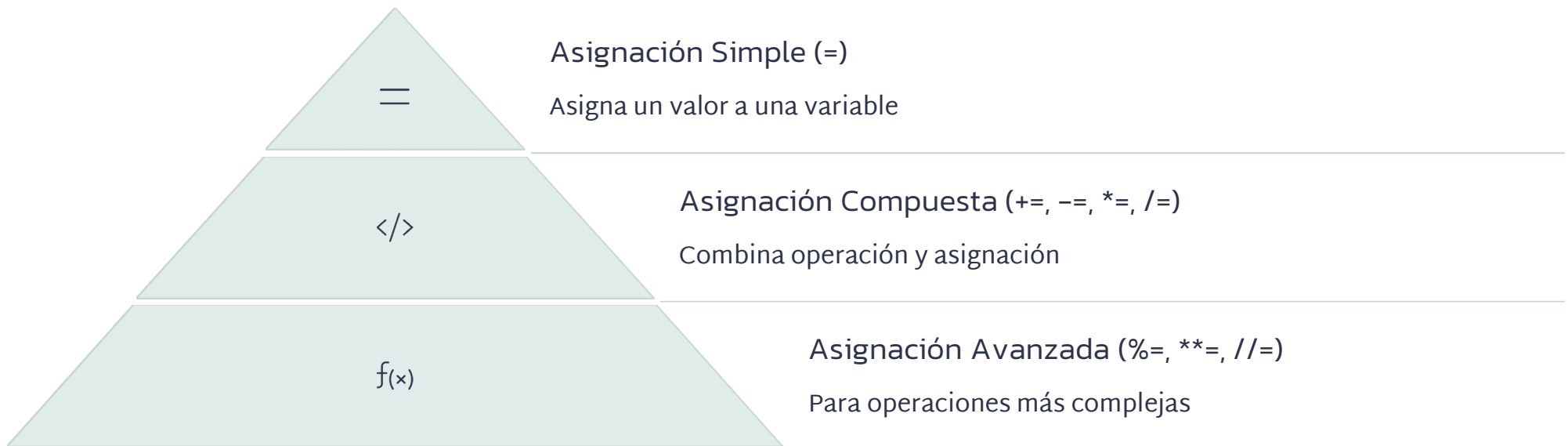
`multiplicacion = a * b` `# multiplicacion = 60`

`division = a / b` `# division = 2.4 (resultado decimal)`

`division_entera = a // b` `# division = 2 (resultado entero)`

`modulo = a % b` `# modulo = 2`

Operadores de Asignación



Los operadores de asignación permiten dar un valor a una variable o modificarlo. Python posee ocho operadores de asignación diferentes: un operador de asignación simple y siete operadores de asignación compuestos.

El operador de asignación simple es el símbolo igual (=) y las operaciones tienen la sintaxis: `variable = expresión`.

Los operadores compuestos como += (equivalente a `variable = variable + expresión`) permiten actualizar el valor de una variable de forma más concisa.

Operador de asignación básico =

El operador de asignación básico (=) se utiliza para asignar un valor a una variable. Por ejemplo:

```
numero = 10
```

En este ejemplo, numero es una variable a la que se le asigna el valor 10.

El operador de asignación también se utiliza para actualizar el valor de una variable. Por ejemplo:

```
a = 5; # Asignación inicial
```

```
a = 10; # Actualización del valor
```


Operadores de asignación compuestos

Operador +=

El operador += se utiliza para sumar el valor de la derecha a la variable y asignar el resultado a la variable.

contador = 5

contador += 3 # Equivalente a contador = contador + 3

Después de esta operación, contador será igual a 8.

Operador -=

El operador -= se utiliza para restar el valor de la derecha a la variable y asignar el resultado a la variable.

total = 100

descuento = 20

total -= descuento # Equivalente a total = total - descuento

Después de esta operación, total será igual a 80.

Operador *= y /=

Estos operadores se utilizan para multiplicar (*) y dividir (/) la variable por el valor de la derecha y asignar el resultado a la variable, respectivamente.

precio = 100

descuento = 20

precio /= (100 - descuento) / 100 # Equivalente a precio = precio / ((100 - descuento) / 100)

Operador //=

El operador //= se utiliza para realizar una división entera y asignar el resultado a la variable.

numero = 25

numero //= 4 # Equivalente a numero = numero // 4, el resultado es 6

Después de esta operación, numero será igual a 6.

Operador %= y **=

Estos operadores se utilizan para calcular el módulo (%) y la potencia (**) de la variable y asignar el resultado a la variable, respectivamente.

numero = 10

numero %= 3 # Equivalente a numero = numero % 3

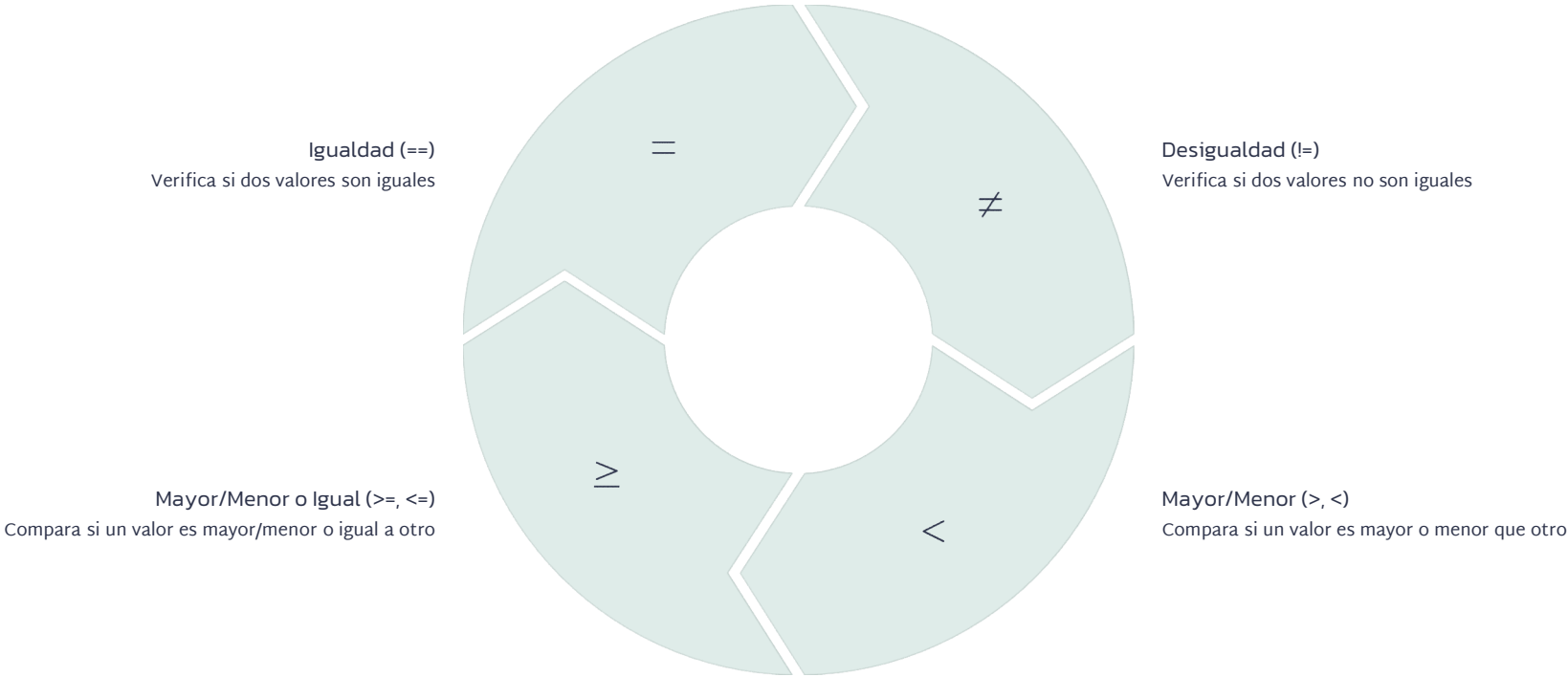
base = 2

exponente = 3

base = **exponente** # Equivalente a base = base ** exponente

Operadores Relacionales y Lógicos

Los operadores relacionales son símbolos que se utilizan para comparar dos valores o expresiones. El resultado de la evaluación con estos operadores puede ser True, si la comparación es cierta, o False, si la comparación es falsa.



Los operadores lógicos permiten conectar expresiones de comparación: **and** (verdadero si ambas condiciones son verdaderas), **or** (verdadero si al menos una condición es verdadera). **not** (invierte el valor lógico).

Operador	Descripción	Ejemplo
and	Operador «y»	a and b
or	Operador «o»	a or b
not	Operador «no»	not a

Igualdad ==

El operador de igualdad (==) se utiliza para verificar si dos valores son iguales.

a = 5

b = 5

es_igual = (a == b) # True, ya que a y b son iguales

Desigualdad !=

El operador de desigualdad (!=) se utiliza para verificar si dos valores no son iguales.

a = 5

b = 3

no_es_igual = (a != b) # True, ya que a no es igual a b

Mayor que >

El operador mayor que (>) se utiliza para verificar si un valor es mayor que otro.

a = 10

b = 5

es_mayor = (a > b) # True, ya que a es mayor que b

Menor que <

El operador menor que (<) se utiliza para verificar si un valor es menor que otro.

a = 3

b = 7

es_menor = (a < b) # True, ya que a es menor que b

Mayor o igual que >=

El operador mayor o igual que (>=) se utiliza para verificar si un valor es mayor o igual que otro.

a = 10

b = 10

es_mayor_igual = (a >= b) # True, ya que a es igual a b

Menor o igual que <=

El operador menor o igual que (<=) se utiliza para verificar si un valor es menor o igual que otro.

a = 5

b = 7

es_menor_igual = (a <= b) # True, ya que a es menor que b

Operador and

El operador and se utiliza para combinar dos condiciones, y la expresión resultante será True solo si ambas condiciones son True.

a = 5

b = 10

c = 15

resultado = (a < b) and (b < c) # True, ya que ambas condiciones son True

Operador or

El operador or se utiliza para combinar dos condiciones, y la expresión resultante será True si al menos una de las condiciones es True.

a = 5

b = 10

c = 3

resultado = (a > b) or (b < c) # False, ya que ninguna de las condiciones es True

Operador not

El operador not se utiliza para negar una condición, es decir, convierte True en False y viceversa.

verdadero = True

falso = not verdadero # falso es False

numero = 5

es_mayor_a_10 = numero > 10

no_es_mayor_a_10 = not es_mayor_a_10

Ejemplos de uso

Combinación con and

```
edad = 25
tiene_licencia = True
puede_conducir = (edad >= 18) and tiene_licencia # True
```

Combinación con or

```
esta_lloviendo = False
es_frio = True
quiere_quedarse_en_casa = esta_lloviendo or es_frio # True
```

Negación con not

```
es_dia = True
es_noche = not es_dia # False
```

Combinación de operadores lógicos

Es posible combinar varios operadores lógicos en una sola expresión para crear condiciones más complejas.

```
numero = 15
es_multiplo_de_3 = (numero % 3 == 0)
es_multiplo_de_5 = (numero % 5 == 0)
es_multiplo_de_3_o_5 = es_multiplo_de_3 or es_multiplo_de_5 # True, ya que 15 es múltiplo de 3
es_multiplo_de_3_y_5 = es_multiplo_de_3 and es_multiplo_de_5 # False, ya que 15 no es múltiplo de 3 y 5 simultáneamente
no_es_multiplo_de_3 = not es_multiplo_de_3 # False
```

Precedencia de operadores lógicos

Cuando se combinan múltiples operadores lógicos en una expresión, es importante entender su precedencia para evaluar correctamente la expresión.

La precedencia de operadores lógicos en Python sigue este orden:

not

and

or

Por lo tanto, en una expresión con and, and se evalúa antes que or.

Sin embargo, no conviene abusar de esto a la hora de escribir nuestro código. Mejor usar paréntesis si mejora la legibilidad.

Operadores de identidad

Los operadores de identidad evalúan si las referencias de dos variables apuntan al mismo objeto. Python cuenta con dos operadores de identidad:

Operador	Descripción	Ejemplo
is	Operador es	a is b
is not	Operador no es	a is not b

El operador `is` tiene la sintaxis `variable is variable`. Es un operador booleano que comprueba si las referencias de las variables que aparecen a ambos lados del operador apuntan al mismo objeto. Si ambas referencias apuntan al mismo objeto devuelve `True` y en caso contrario devuelve `False`.

Operador is

El operador `is` se utiliza para verificar si dos variables se refieren al mismo objeto en la memoria.

```
a = [1, 2, 3]
```

```
b = a
```

```
es_igual = (a is b) # True, ya que a y b se refieren al mismo objeto
```

En este ejemplo, tanto `a` como `b` apuntan al mismo objeto en la memoria, por lo que la expresión `a is b` es `True`.

Operador is not

El operador `is not` se utiliza para verificar si dos variables NO se refieren al mismo objeto en la memoria.

```
x = 10
```

```
y = 10
```

```
no_es_igual = (x is not y) # False, ya que x e y se refieren al mismo objeto
```

En este ejemplo, tanto `x` como `y` apuntan al mismo objeto en la memoria (en Python, los enteros pequeños se almacenan en la misma ubicación), por lo que la expresión `x is not y` es `False`.

Ejemplos de uso

Verificación de identidad de variables

```
x = 100
y = x
son_iguales = (x is y) # True, ya que x e y son el mismo objeto en memoria
```

Verificación de identidad de listas

```
lista1 = [1, 2, 3]
lista2 = [1, 2, 3]
lista3 = lista1
son_iguales = (lista1 is lista2) # False, ya que lista1 y lista2 no son el mismo objeto
son_iguales2 = (lista1 is lista3) # True, ya que lista1 y lista3 son el mismo objeto
```

Verificación de identidad de objetos

```
class MiClase:...

objeto1 = MiClase()
objeto2 = MiClase()
objeto3 = objeto1
son_iguales = (objeto1 is objeto2) # False, ya que objeto1 y objeto2 no son el mismo objeto
son_iguales2 = (objeto1 is objeto3) # True, ya que objeto1 y objeto3 son el mismo objeto
```

Operadores de pertenencia

Los operadores de pertenencia, también conocidos como operadores miembro, permiten comprobar si un dato forma parte o no de una colección: una lista, una tupla, una cadena, etc. Aún no hemos visto estos tipos de datos, así que no te preocupes si te sientes desorientado. Las colecciones son agrupaciones de valores y las estudiaremos más adelante.

Operador	Descripción	Ejemplo
in	Operador pertenece	a in b
not in	Operador no pertenece	a not in b

Operador in

El operador in se utiliza para verificar si un valor está presente en una secuencia.

```
frutas = ["manzana", "banana", "cereza"]
```

```
es_banana = "banana" in frutas # True
```

```
es_uva = "uva" in frutas # False
```

En este ejemplo, estamos verificando si "banana" está presente en la lista frutas, lo cual es True. También verificamos si "uva" está presente, lo cual es False.

Operador not in

El operador not in se utiliza para verificar si un valor NO está presente en una secuencia.

```
frutas = ["manzana", "banana", "cereza"]
```

```
no_es_uva = "uva" not in frutas # True
```

```
no_es_manzana = "manzana" not in frutas # False
```

En este ejemplo, estamos verificando si "uva" NO está presente en la lista frutas, lo cual es True. También verificamos si "manzana" NO está presente, lo cual es False.

Ejemplos de uso

Verificación de elementos en una lista

```
colores = ["rojo", "verde", "azul"]
```

```
es_azul = "azul" in colores # True
```

```
no_es_amarillo = "amarillo" not in colores # True
```

Verificación de caracteres en una cadena

```
nombre = "Luis"
```

```
tiene_a = "a" in nombre # False
```

```
no_tiene_z = "z" not in nombre # True
```

Verificación de elementos en un conjunto

```
vocales = {"a", "e", "i", "o", "u"}
```

```
es_a_vocal = "a" in vocales # True
```

```
no_es_y = "y" not in vocales # True
```

Operadores a nivel de bit

Los operadores a nivel de bit se utilizan para comparar números enteros en su representación binaria, es decir, operan sobre cada uno de los bits del entero sobre el que se aplican, y devuelven el número entero correspondiente al resultado binario. Por ejemplo, cualquier operación binaria sobre los enteros 1 y 2 se realizará realmente entre los números binarios 01 y 10, que son la representación binaria de dichos enteros.

Operador	Descripción	Ejemplo
& (AND)	Operador «y» binario	a & b
(OR)	Operador «o» binario	a b
^ (XOR)	Operador «o» exclusivo binario	a ^ b
~ (complemento)	Operador «no» binario	~a
<<	Operador desplazamiento izquierdo	a << b
>>	Operador desplazamiento derecho	a >> b

Operador AND &

El operador AND (&) compara cada bit de dos operandos y devuelve 1 si ambos bits son 1, de lo contrario devuelve 0.

#Ejemplo de operador AND

a = 10 # 1010 en binario

b = 3 # 0011 en binario

resultado = a & b

print(resultado) # Salida: 2

#Explicación: 1010 & 0011 = 0010 (decimal 2)

Operador OR |

El operador OR (|) compara cada bit de dos operandos y devuelve 1 si al menos uno de los bits es 1.

#Ejemplo de operador OR

a = 10 # 1010 en binario

b = 3 # 0011 en binario

resultado = a | b

print(resultado) # Salida: 11

#Explicación: 1010 | 0011 = 1011 (decimal 11)

Operador XOR ^

El operador XOR (^) compara cada bit de dos operandos y devuelve 1 si exactamente uno de los bits es 1, pero no ambos.

#Ejemplo de operador XOR

a = 10 # 1010 en binario

b = 3 # 0011 en binario

resultado = a ^ b

print(resultado) # Salida: 9

#Explicación: 1010 ^ 0011 = 1001 (decimal 9)

Operadores de desplazamiento

Los operadores de desplazamiento mueven los bits de un número hacia la izquierda (<<) o hacia la derecha (>>) en la cantidad especificada.

#Ejemplo de desplazamiento izquierda

a = 5 # 101 en binario

resultado = a << 2

print(resultado) # Salida: 20

#Explicación: $101 \ll 2 = 10100$ (decimal 20)

#Ejemplo de desplazamiento derecha

a = 20 # 10100 en binario

resultado = a >> 2

print(resultado) # Salida: 5

#Explicación: $10100 \gg 2 = 101$ (decimal 5)

Operador de complemento ~

El operador de complemento (~) invierte todos los bits de un número.

#Ejemplo de operador de complemento

a = 5 # 101 en binario

resultado = ~a

print(resultado) # Salida: -6

#Explicación: $\sim 101 = -110$ (en complemento a dos, -6 en decimal)

Precedencia de los operadores

Hasta ahora hemos hablado sobre los operadores que ofrece Python y visto ejemplos de aplicación de cada uno de ellos según su tipo, pero ¿qué ocurre si nos encontramos con una expresión con más de un operador? ¿En qué orden se deben evaluar? Los operadores tienen un orden de prioridad preestablecido para determinar qué operaciones se deben evaluar primero en caso de que una expresión sea ambigua. Supongamos que tenemos la expresión $6 + 2 * 3$.

¿Qué operación se realiza primero, la suma o la multiplicación? El resultado de esta expresión es 12 porque el operador *tiene una prioridad superior al operador +*. Por tanto, en primer lugar, se realiza la multiplicación $2 * 3$ y, posteriormente, se suma el valor 6, lo cual da como resultado el valor 12. En la tabla 6.14 se presenta el orden de precedencia de los operadores en Python.

No obstante, es posible alterar el orden de prioridad de estos operadores haciendo uso de los paréntesis. Estos elementos marcan el orden en el que se realizan las operaciones y resultan muy útiles para ver fácilmente cómo se agrupan dichas operaciones. En el ejemplo anterior, si quisiéramos que en primer lugar se realizara la operación suma, tendríamos que indicarlo mediante el uso de paréntesis de la siguiente manera: $(6 + 2) * 3$, lo que daría como resultado el valor 24.

Operador	Descripción	Prioridad
**	Potencia	1
~	Operador «no» binario	2
*, /, %, //	Multiplicación, cociente de la división, resto de la división entera y cociente de la división entera	3
+, -	Suma y resta	4
>> <<	Operador desplazamiento derecho y operador desplazamiento izquierdo	5
&	Operador «y» binario	6
^	Operador «o» exclusivo binario y operador «o» binario	7
<=, <, >, >=	Operadores de comparación: menor o igual que, menor que, mayor que y mayor o igual que	8
== != ,	Igual a y distinto a	9
=, %= /= //=-=, += *= **=	Operadores de asignación: simple, módulo, división, división entera, resta, suma, multiplicación, exponencial	10
is is not	Operadores de identidad: «es» y «no es»	11
in not in	Operadores de pertenencia: «pertenece» y «no pertenece»	12
not or and	Operadores lógicos: «no», «o» e «y»	13

Ejercicios propuestos

Mi coche gasta 5,5 litros cada 100 km y mi trabajo se encuentra a 15 kilómetros de casa. ¿Cuánto me gastaré en gasolina en 20 días laborables si el precio es de 1,12 €/l?

En enero del año actual tenía una cuenta con 3000 €. Si cobro 1100 € mensuales y tengo unos gastos fijos al mes de 435 €, ¿a cuánto ascienden mis gastos extra mensuales si a final de año mi cuenta tiene un total de 6000 €?

Tengo 50 € para comprar una camisa. Si la camisa cuesta 35 € y tiene un descuento del 10 %, ¿cuánto dinero tendré después de comprar la camisa?

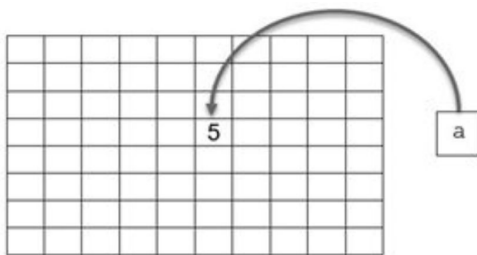
Variables y Tipos de Datos

La memoria del ordenador

En un lenguaje de programación, una variable es un término que representa un espacio en la memoria del ordenador. Si queremos trabajar con un valor entero, ese tipo de dato necesita de un tamaño de memoria determinado. Si lo que nos interesa es operar con un número real, dependerá del nivel de precisión y requerirá un tamaño diferente. Para una secuencia de caracteres, que denominamos «cadena» (*string* en inglés), necesitaremos un espacio equivalente al de la longitud de dicha cadena. En definitiva, **una variable no es sino un nombre que el programador usa para referenciar la posición de un hueco en la memoria RAM del ordenador.**

La memoria del ordenador es como un casillero. Consiste en una matriz de celdas de tamaño fijo. Cada celda puede almacenar 8 bits y todas las celdas ocupan posiciones consecutivas, por lo que podemos referenciarlas mediante su posición de forma directa. Un tamaño de 8 bits se denomina «byte».

Cuando Python ejecuta una asignación, es decir, el almacenado de un valor a una variable (por ejemplo, $a=5$), **el intérprete busca una casilla libre en memoria, almacena ahí el valor y apunta la dirección de esa casilla en una tabla** para recordar que cuando aparezca esa variable de nuevo, debe ir a dicha dirección. Si la variable aparece en una expresión (por ejemplo $\text{print}(a * 2)$), sabe dónde tiene que ir a buscar el valor asociado a dicha variable para resolver la operación. Afortunadamente, **Python se encarga de reservar la memoria y de tomar el espacio necesario según el tipo de dato que asignemos a una variable.** El código siguiente nos permite conocer la dirección de memoria que Python ha asociado a una variable (usando la función `id()`), así como el número de bytes necesarios para alojar sus valores si es de tipo entero (usando el método `bit_length()`).



La línea de código siguiente guarda el valor 5 en memoria:

```
a = 5
```

```
a = 5
id(a)
140733324637136
a.bit_length()
3
```

Cada tipo de dato requiere un número de bytes diferente, como veremos más adelante. Las colecciones requieren de una cantidad de celdas de memoria (bytes) que variará según el tipo de dato y el número de elementos que guarde. Como los elementos pueden estar repartidos por distintas ubicaciones (en función de los espacios disponibles), Python guarda para la variable la dirección de la «cabecera» de la colección, que es, por decirlo así, el inicio de la colección. Profundizaremos en estos tipos de datos contenedores más adelante.

Identificadores

Llamamos identificadores a los nombres que damos a variables, funciones, clases y métodos. Tenemos libertad para decidir cómo nombrar a una nueva variable, siempre que cumplamos que la cadena de caracteres contenga caracteres válidos. Su longitud es ilimitada, aunque puedes imaginar que nombres muy largos pueden hacer difícil de entender el código. Un identificador puede usar letras mayúsculas y minúsculas de la «A» a la «Z», el guion bajo «_» y, excepto para el primer carácter, dígitos del 0 al 9. Además, para Python3, se añade la posibilidad de ampliar los caracteres posibles con cualquier letra Unicode. Así, en nuestro primer programa podríamos haber usado el identificador año_nac en lugar de anio_nac. Podemos, incluso, usar tildes. En la tabla siguiente se muestran algunos ejemplos de identificadores válidos e inválidos.

Válidos	Inválidos
Peso	3dimensiones
órgano	precio medio
res3	Valor-máximo
FactorDestacado	365grados
cañón34híbrido	1ºlista
mim_ele	cabeza/pie
_propSup	Total\$
privado	orden:inv
	<i>final</i>
	saldo.cuenta

Esta posibilidad de añadir tildes y otros caracteres Unicode facilita escribir nombres, sea cual sea el idioma del programador. Pero esta gran flexibilidad que ofrece Python a la hora de crear identificadores es, desde mi humilde opinión, poco deseable. ¿Por qué? Porque la gran mayoría de lenguajes de programación no aceptan identificadores distintos a letras ASCII de la A a la Z (sin tildes ni eñes), dígitos del 0 al 9 y guion bajo. Es bueno crear hábitos y un estilo acorde a las recomendaciones más extendidas. Te resumimos aquí algunas de ellas:

- No uses caracteres Unicode. Otros lenguajes no los admiten y pueden limitar la portabilidad de tu código.
- Usa identificadores claros, no muy largos. Un nombre de variable como *media* es mejor a *valor_medio_de_elementos*.
- Cuando uses varias palabras en un mismo identificador, utiliza guion bajo o intercambia mayúsculas y minúsculas de manera consistente. Por ejemplo, algunos programadores utilizan solo guiones bajos para variables y funciones: actualizar_renta(), saldo_cuenta. Otros programadores prefieren el estilo «CamelCase»: ActualizarRenta(), saldoCuenta. También hay quien opta por una combinación de ambas, comenzando las variables siempre con minúscula y usando guiones en ellas, y empezando con mayúscula y estilo CamelCase para nombres de clases, funciones y métodos.

En Python existen ciertas reglas y buenas prácticas para seguir.

Algunas de ellas son obligatorias (impuestas por el lenguaje), y otras son simplemente convenciones que puedes seguir.

Reglas Específicas

Case Sensitive: Los nombres de variables en Python son sensibles a mayúsculas y minúsculas. Por ejemplo, variable, Variable y VARIABLE son consideradas diferentes.

Inicio de Nombre: El nombre de la variable debe comenzar con una letra (a-z, A-Z) o un guión bajo (_). No debe comenzar con números ni otros caracteres. Recomendado que las variables empiecen con minúsculas y las clases con mayúsculas.

```
variable = 42
```

```
_variable_privada = "Python"
```

```
1variable = "Error" # Incorrecto, no debe comenzar con un número
```

Caracteres Permitidos: Los nombres de variables pueden contener letras (a-z, A-Z), dígitos (0-9) y guiones bajos (_). No se permiten otros caracteres especiales como espacios, signos de puntuación o símbolos.

```
mi_variable = "Python"
```

```
edad3 = 30
```

```
mi_variable@ = "Error" # Incorrecto, no se permite el símbolo @
```

Palabras Reservadas: No se pueden utilizar palabras reservadas de Python como nombres de variables. Por ejemplo, if, else, for, while, entre otras.

```
if = 10 # Incorrecto, if es una palabra reservada  
mi_if = 10 # Correcto, añadiendo prefijo o sufijo a la palabra reservada
```

Legibilidad y Significado: Es importante elegir nombres de variables que sean descriptivos y reflejen su uso en el contexto del programa. Esto mejora la legibilidad y comprensión del código.

```
total_compras = 100 num = 100 # Menos descriptivo, se desconoce el propósito
```

Convenciones de Estilo

Python PEP 8 (Guía de Estilo para el Código Python) sugiere convenciones adicionales para mejorar la legibilidad del código:

snake_case: Para nombres de variables, funciones y métodos. Por ejemplo, `mi_variable`, `calcular_total()`.

CamelCase: Para nombres de clases en Python. Por ejemplo, `MiClase`, `MiClaseEspecial`.

#Ejemplos de snake_case

```
nombre_completo = "Luis Pérez"
```

```
calcular_total = lambda x, y: x + y
```

#Ejemplos de CamelCase (para nombres de clases)

```
class MiClase:
```

```
    def init(self):
```

```
        self.atributo = None
```

Mayúsculas para el caso de constantes, separadas por guiones bajos, siguiendo el estilo `CONSTANTE_DE_EJEMPLO`

Tipos de datos

Python cuenta con un interesante surtido de tipos de datos de manera directa. Corresponden a tipos reales, enteros, cadenas de caracteres, booleanos, complejos, tuplas, conjuntos y diccionarios. Estos tipos básicos suelen ser suficientes para implementar soluciones válidas en la mayoría de los casos. Sin embargo, es posible trabajar con tipos de datos adicionales, proporcionados tanto por la biblioteca estándar como por otras bibliotecas. Altas precisiones de números reales suelen ser necesarias, por ejemplo, en cálculo científico, por lo que en esos casos optaríamos por los tipos de datos que facilitan bibliotecas como NumPy. Examinemos ahora los tipos básicos principales de Python con algunos ejemplos:

Tipo	Descripción	Ejemplos
None	Este tipo especial indica que no hay valor alguno (por lo que no toma rango de valores en el tipo). Se utiliza para indicar que una variable o parámetro no tiene valor asignado. Se interpreta también como valor False en una expresión lógica.	precio = None
int	Tipo de dato entero. Representa valores enteros, positivos y negativos, sin límite de rango (solo dependiente de la memoria disponible). Funciones para trabajar con enteros: - abs(): Devuelve el valor absoluto de un número entero - pow(base, exponente): Calcula la potencia de un número - divmod(a, b): Devuelve el cociente y el resto de la división de a entre b	num_hijos = 3 saldo = -3000

bool	Tipo de dato lógico o booleano. Puede tomar solo dos posibles valores: False (falso) o True (verdadero). Realmente se almacenan como un valor 0 o 1 respectivamente.	es_mayor = edad > 18 activado = False
float	Tipo de dato real. Su nivel de precisión está en función de los tamaños de su parte entera y parte decimal, y de la implementación interna del intérprete de Python que utilizemos. Los literales separan decimales con un punto y pueden usar notación científica con la letra «e» o «E» para indicar el exponente. Una variable real también puede tomar el valor nan (que significa que es un valor inválido) o inf (que significa «infinito»).	Pi = 3.141592 menor = 7e- 10 valor_p = .0002 ratio = 342/87 infinito_positivo = float('inf') nan_valor = float('nan')
complex	Tipo de dato complejo, con dos valores reales: uno es la parte real y otro la imaginaria. En lugar de usar la letra «i» para la imaginaria, Python expresa este componente con la letra «j» en minúscula o mayúscula.	comp = 3+5j punto3d = 8.4-1e100j
str	Una cadena de texto es una secuencia de caracteres. Tienen un tamaño dado, que podemos conocer con la función len(). Dedicaremos un capítulo a este tipo de datos. Las cadenas en Python admiten caracteres especiales para representar saltos de línea (\n), tabulaciones (\t), comillas dentro de cadenas (\", \'), entre otros. Funciones para trabajar con cadenas: - startswith(prefix): Devuelve True si la cadena empieza con el prefijo dado. - endswith(suffix): Devuelve True si la cadena termina con el sufijo dado. - isalpha(): Devuelve True si todos los caracteres de la cadena son letras. - isdigit(): Devuelve True si todos los caracteres de la cadena son dígitos.	nombre = "Pedro" apellido = "Gómez" id = nombre + " " + apellido len(id) cadena_con_salto = "Primera línea \n Segunda línea" print(cadena_con_salto)
tuple	Una tupla es una secuencia de valores de cualquier tipo (en realidad, una secuencia arbitraria de objetos). A diferencia de las listas, sus valores no pueden cambiarse.	coord = (334, 87.156) ser = (nom, ape, peso) prod = ("Tomate", 1.55)
list	Es una secuencia arbitraria de objetos que podemos manipular y variar de formas diversas.	precios = [2, 4.5, 65] precios[2] = 64 * 2
set	Es como una lista, pero los elementos no pueden repetirse ni indexarse. Podemos añadir y extraer elementos del conjunto, pero no podemos tener dos elementos iguales.	conj2 = {1, "hola", (5, 6)}
dict	Es otro contenedor de objetos, pero asocia a cada elemento una «clave» con la que accede al mismo	d = {'precio': 34, 'producto': 'consola', 'stock': 2} d['stock'] = d['stock'] -1

Los enteros, reales, booleanos, complejos, cadenas y tuplas son tipos de datos «inmutables». Ello significa que cualquier cambio en su valor implica, generalmente, una nueva ubicación en memoria. Las listas, tuplas, conjuntos, diccionarios y cadenas merecerán capítulos específicos para profundizar en su utilidad y versatilidad.

Literales

Un literal es un valor fijo, explícito, concreto, que indicamos en nuestro código. La cadena 'amistad' es un literal, el valor real 0.34 es un literal, el entero 2019 es otro literal. En la jerga de Python, un literal es una combinación de caracteres que el intérprete reconoce como una representación de un valor de objeto válido. En la lista no exhaustiva de tipos de datos que hemos mostrado se observa que a una variable se le asigna siempre una «expresión». Una expresión es toda combinación válida de operadores, variables, llamadas a funciones y, por supuesto, literales. Algunos ejemplos más de literales para cada tipo de dato:

- **Enteros (int):** 0, 2, 56, -2, -20, 1
- **Reales (float):** -0.23, 34E+45, 8923.2342, 0.00003
- **Complejos (complex):** 2+5j, 0+4j, 8j, 23.3+.9j
- **Cadenas (str):** "", "hola", "el resultado es:"
- **Listas (list):** [], [1, 2], ['a', 'b', 'c', 'd'], ['Pedro', 34, 'Luis', 35, 'Paula', 18, 'Julia', 12]
- **Tuplas (tuple):** (), (3,), (3, 4, 6, 7), ('coordenadas', 23423.34, 763.424)
- **Diccionarios (dict):** {'a': 3, 'b': 8, 'c': .5}, {'primero': 23, 'segundo': 'móvil', 'tercero': (3, 5)}
- **Conjuntos (set):** {1,2,3}, {'fruta', 'carne', 'verdura', 'pescado'}, {3, 'b', 3.14}

La vida de una variable

Una variable es un identificador asociado a un espacio en memoria donde almacenar datos. En Python, todo son «objetos». Hablaremos con más detalle sobre los objetos en el capítulo correspondiente. Una variable es también un objeto. En Python, todo objeto tiene una identidad, un tipo y un valor. La identidad nunca cambia, y podemos averiguarla, como ya hemos visto, con la función `id()`. La identidad es el número de casilla en memoria donde comienza el espacio para alojar el valor. Dicho espacio dependerá del tipo de dato asociado. Podemos comparar la identidad de dos objetos usando el operador `is`. Los valores de un objeto pueden cambiar en el caso de los objetos «mutables», o permanecer invariables en el caso de los objetos «inmutables». La mutabilidad dependerá del tipo asociado.

Inicialización y destrucción

Vamos a continuar hablando de variables, en lugar de objetos, aunque lo último sería más correcto. A diferencia de otros lenguajes, no es necesario «inicializar» las variables, es decir, asignar un valor por defecto al crearlas.

Las variables se crean cuando se usan por primera vez y tampoco es necesario destruirlas de forma explícita, aunque podemos forzarlo usando la función `del()`. Destruir una variable significa que su identificador deja de representar valor alguno; la variable ya no está definida y el espacio de memoria que ocupaba queda disponible para otros usos. Cuando una variable alcanza un determinado estado de «inutilidad», el recolector de basura de Python se encarga de eliminarla. El tipo de una variable cambia al asignar un valor de distinto tipo al actual:

```
a = 4
```

```
type(a)
```

```
int a = a * 0.5
```

```
type(a)
```

```
float
```


Casting

Aunque Python determina el tipo de manera dinámica, podemos obligar a que el resultado de una expresión se almacene bajo un tipo determinado por el programador. A esta especificación explícita de tipo se le denomina casting. Por ejemplo, la asignación `a = float(4)` creará la variable `a` con tipo real, aunque el valor de origen sea un entero. El casting se realiza mediante «funciones de construcción» que crean objetos (variables) nuevos inicializándolos a partir del valor indicado. Las funciones de construcción más utilizadas para el casting son las siguientes:

- **int()**: crea un entero a partir de un literal o variable de tipo entero, un literal o variable de tipo real (mediante redondeo hacia abajo), o un literal o variable de tipo cadena (siempre que esa cadena represente un entero válido).
- **float()**: crea un real a partir de un literal o variable de tipo entero, un literal o variable de tipo real o un literal o variable de tipo cadena válido.
- **str()**: crea una cadena a partir de distintos tipos de datos, entre los que se incluyen enteros, reales y otras cadenas.
- **bool()**: convierte un valor a booleano. Es importante tener en cuenta que no todas las conversiones son posibles. En caso de hacer una conversión no válida, se pueden producir pérdida de información (o incluso generar un error).

Para ver el efecto del casting, lo mejor es probarlo sobre la terminal interactiva de Python:

```
#Conversión de flotante a entero
```

```
int(4.5)
```

```
4
```

```
#Conversión de cadena a entero
```

```
int('23')
```

```
23
```

```
#Conversión de booleano a entero
```

```
booleano = True
```

```
entero = int(booleano)
```

```
#Conversión de entero a flotante
```

```
float(5)
```

```
5.0
```

```
#Conversión de cadena a flotante
```

```
float('3.14')
```

```
3.14
```

```
#Conversión de booleano a flotante
```

```
booleano = True
```

```
flotante = float(booleano)
```

```
#Conversión de entero a cadena
```

```
entero = 10
```

```
cadena = str(entero)
```

```
#Conversión de flotante a cadena
```

```
flotante = 3.14
```

```
cadena = str(flotante)
```

```
#Conversión de booleano a cadena
```

```
booleano = True
```

```
cadena = str(booleano)
```

```
#Conversión de entero a booleano
```

```
entero = 0
```

```
booleano = bool(entero)
```

Ámbito de una variable

El ámbito o scope de una variable se refiere a **la parte del programa donde esa variable es accesible**. En Python, hay dos tipos principales de alcance de variables: **local y global**.

Variables locales: Son aquellas definidas dentro de una función. Su alcance está limitado a esa función y no son accesibles fuera de ella. Si intentamos acceder a una variable local desde fuera de la función, Python generará un error.

```
def mi_funcion():  
    variable_local = "Hola Mundo"  
    print(variable_local)  
mi_funcion()
```

```
#print(variable_local) # Esto generaría un error
```

En este ejemplo, `variable_local` está definida dentro de la función `mi_funcion` y solo es accesible dentro de esa función.

Variables globales: Son aquellas definidas fuera de cualquier función o en un ámbito global. Pueden ser accesibles desde cualquier parte del programa (tanto dentro como fuera de las funciones).

```
variable_global = "Soy global"  
def imprimir_global():  
    print(variable_global)  
imprimir_global() # Resultado: Soy global  
print(variable_global) # También es accesible fuera de la función
```

En este caso, `variable_global` está definida fuera de la función `imprimir_global`, por lo que es una variable global y puede ser accesible desde cualquier parte del programa.

Uso de la palabra clave global

En algunos casos, es posible que necesitemos **modificar una variable global desde dentro de una función**. Para hacerlo, necesitamos utilizar la palabra clave global.

```
variable_global = "Soy global"
```

```
def modificar_global():
```

```
    global variable_global
```

```
    variable_global = "Modificado dentro de la función"
```

```
    print(variable_global)
```

```
modificar_global() # Resultado: Modificado dentro de la función
```

```
print(variable_global) # Ahora la variable global ha sido modificada
```

Al utilizar `global variable_global` dentro de la función `modificar_global`, le estamos indicando a Python que queremos modificar la variable global `variable_global`.

Tipos valor y tipos referencia en Python

En Python, los tipos de datos pueden ser divididos en dos categorías principales: **tipos de valor** y **tipos de referencia**

Esto es algo común en muchos lenguajes de programación. Entender las diferencias entre ambas es muy importante.

Tipos valor

Los tipos valor en Python son aquellos cuyas variables **almacenan directamente el valor real**. Esto significa que cuando asignamos una variable a otro valor, se crea una nueva copia de ese valor en la memoria.

Los tipos de valor más comunes en Python son los tipos básicos como enteros, flotantes, cadenas y booleanos. Por ejemplo:

#Asignación de un entero a una variable

```
x = 10
```

#se crea otra variable que toma el valor de x.

```
y = x
```

#ambas variables comparten la misma posición en memoria

```
print(id(x))
```

```
print(id(y))
```

```
print(y is x) #True
```

#si modificamos el valor de la variable y, x e y tienen valores independientes en la memoria

```
y = 9
```

```
print(y is x) #False
```

```
print(id(x))
```

```
print(id(y))
```

#Cuando modificamos x o y, cada variable tiene su propio espacio en la memoria para almacenar su valor.

Tipos referencia

Los tipos referencia son aquellos cuyas variables **almacenan una referencia** (normalmente la dirección de memoria) al objeto real en lugar del valor real en sí mismo.

Esto significa que cuando asignamos una variable a otra, ambas variables apuntan al mismo objeto en la memoria. Los tipos de referencia comunes incluyen listas, diccionarios, conjuntos y objetos personalizados. Un ejemplo:

#Asignación de una lista a una variable

```
lista1 = [1, 2, 3]
```

#lista2 apunta al mismo objeto en la memoria que lista1

```
lista2 = lista1
```

#Modificar lista2 también modifica lista1

```
lista2.append(4)
```

```
print("lista1:", lista1) # Resultado: [1, 2, 3, 4]
```

En este ejemplo, cuando modificamos lista2, también se modifica lista1 porque ambas variables apuntan al mismo objeto en la memoria.

Pasando argumentos a funciones

(Lo volveremos a ver en el tema 3: funciones)

Cuando pasamos variables a funciones, **debemos tener en cuenta si son tipos de valor o tipos de referencia**.

Pasando tipos de valor

Los tipos valor no se modifican cuando son pasados como parámetros, porque la función recibe una copia.

```
def duplicar_numero(num):
```

```
    num *= 2
```

```
    print("Dentro de la función:", num)
```

```
x = 10
```

```
duplicar_numero(x)
```

```
print("Fuera de la función:", x) # x sigue siendo 10, no se modificó
```

El valor de x no se modifica fuera de la función duplicar_numero porque es un tipo de valor.

Pasando tipos de referencia

Los tipos referencia sí se modifican cuando son pasados como parámetros, porque la función recibe una copia de la referencia, que apunta a los mismos datos.

```
def modificar_lista(lista):
```

```
    lista.append("Nuevo elemento")
```

```
    print("Dentro de la función:", lista)
```

```
mi_lista = [1, 2, 3]
```

```
modificar_lista(mi_lista)
```

```
print("Fuera de la función:", mi_lista) # mi_lista se modifica
```

En este caso, mi_lista se modifica fuera de la función porque es un tipo de referencia.

Introspección

Ya hemos revelado que en Python una variable es un objeto. Un objeto alberga, además del valor que almacena, una serie de «métodos». Un método es una función que opera sobre el objeto. Según el tipo de dato de una variable, Python predefine un conjunto de métodos que facilitan la manipulación de ese tipo de dato. Sobre una cadena nos puede interesar eliminar espacios al final, o buscar una subcadena. Para las listas tenemos métodos que permiten añadir y eliminar elementos, por ejemplo. Los métodos son muy útiles para trabajar con nuestros datos. De todo esto hablaremos más adelante, basta con que asimiles esta noción de objeto como una variable con métodos asociados.

Los objetos de Python proporcionan, además de métodos propios del tipo de datos que representan, un grupo de métodos y atributos que facilitan conocer mejor nuestras variables: cuáles son los métodos asociados, qué tipo de dato estamos almacenando, y otros elementos que posibilitan examinar la variable en tiempo de ejecución. A esta capacidad de «mirar las tripas» de un objeto en Python se le llama «introspección». Algunas funciones de introspección útiles son las siguientes:

dir(): muestra todos los «miembros» del objeto, es decir, sus atributos y métodos.

variable.__doc__: devuelve una cadena con información de ayuda sobre la variable.

type(): devuelve el tipo de dato asociado a la variable.

id(): devuelve el identificador único asociado a la variable. Generalmente este valor es la dirección de memoria donde se aloja el dato.

Podemos probar estas funciones en la terminal interactiva:

```
a = 34
```

```
print(a.__doc__)
```

```
int([x]) -> integer
```

```
int(x, base=10) -> integer
```

Convert a number or string to an integer, or return 0 if no arguments are given. If x is a number, return x. int (). For floating point numbers, this truncates towards zero.

If x is not a number or if base is given, then x must be a string, bytes, or bytearray instance representing an integer literal in the given base. The literal can be preceded by '+' or '-' and be surrounded by whitespace. The base defaults to 10. Valid bases are 0 and 2-36.

Base 0 means to interpret the base from the string as an integer literal.

```
>>> int('0b100', base=0)
```

```
4
```

```
type(a)
```

```
int
```

```
dir(a)
```

```
['_abs_',
```

```
'_add_',
```

```
'_and_',
```

```
'_bool_',
```

```
'_ceil_',
```

```
'_class_',
```

```
'_delattr_',
```

```
'_dir_',
```

```
'_divmod_',
```

```
'_doc_',
```

```
'_eq_',
```

```
'_float_',
```

```
'_floor_',
```

```
'_floordiv_',
```

```
'_format_',
```

```
'_ge_',
```

```
'_getattrattribute_',
```

```
'_getnewargs_',
```

```
'_gt_',
```

```
'_hash_',
```

```
'_index_',
```

```
'_init_',
```

```
'_init_subclass_',
```

```
'_int_',
```

```
'_invert_',
```

```
'_le_',
```

```
'_lshift_'
```

Ejercicios propuestos

Indica cuáles de los siguientes son identificadores válidos en Python:

```
1erMiembro
libre?
año_nac
SegundoMiembro
puerta.abierta
$saldo$
coche_1
cámara4
import
```

¿Qué tipo de dato usarías para almacenar los siguientes literales?

```
-.28934
[3,5,4]
0.05j
23412354.0
(-34,534)
['ja', 'je', 'ji', 2]
-5463
{'c', 'z', 'j'}
'bienvenido'
12e5
45.1+3.65j
'38 kg.'
```

Indica qué valores aparecerían en pantalla al ejecutar el siguiente código:

```
def calcula()
    a = b * 2
    c = 23
    print (a * c)
b = 5
a = 8
c = 10
print(a)
print(b)
print(c)
```